

Calibración de Probabilidades

predict_proba de scikit-learn no devuelve probabilidades.

Devuelve scores que suman 1, viven entre 0 y 1, y parecen probabilidades. No lo son.

¿Por qué ocurre?

- Discriminación $\neq$ Calibración. Un modelo puede separar perfectamente clases (AUC=1.0) y asignar probabilidades completamente erróneas a cada predicción.
- Random Forest: cada árbol vota 0 o 1 (hojas puras). La "probabilidad" es la fracción de votos. Es un conteo, no una probabilidad.
- Gradient Boosting: empuja log-odds a extremos $\rightarrow$ sigmoid comprime a 0/1. Sobreconfiado en los extremos.
- Logistic Regression: demostró que es sistemáticamente sobreconfiada. Error = $\Theta(d/n)$.

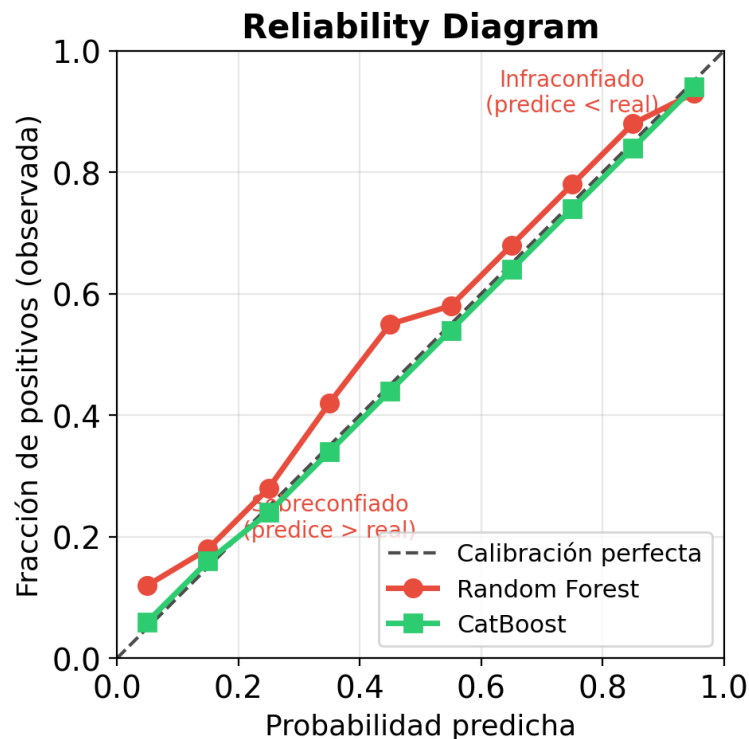
Coste en producción:

- Seguros: modelo dice "12% riesgo" $\rightarrow$ real es 22% $\rightarrow$ infrapricing sistemático.
- Medicina: modelo dice "95% confianza en diagnóstico" $\rightarrow$ real es 74% $\rightarrow$ falsa certeza clínica.
- Fraude: modelo dice "3% fraude" $\rightarrow$ real es 11% $\rightarrow$ transacciones fraudulentas aprobadas.

Nada en el entrenamiento estándar garantiza calibración.

¿Qué es Calibración?

Un modelo calibrado: si dice 80%, el 80% de esos casos son positivos.



Reliability Diagram: la herramienta visual fundamental.

- Diagonal = calibración perfecta.
- Random Forest (rojo): sobreconfiado abajo, infraconfiado arriba. Desviación máx: 0.223.
- CatBoost (verde): pegado a la diagonal. $|Z|=1.88$ (< 1.96). Ya calibrado.

Modelos optimizan discriminación (separar clases), no calibración (probabilidades correctas).

Brier Score combina ambas propiedades. Eso es un problema:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2$$

En binario, se descompone (Spiegelhalter, 1986):

$$\text{MBS}(T) = \underbrace{\frac{1}{N} \sum_{i=1}^N (y_i - p_i)(1 - 2p_i)}_{\text{término de calibración}} + \underbrace{\frac{1}{N} \sum_{i=1}^N p_i(1 - p_i)}_{\text{término de sharpness}}$$

Equivalente a: Brier = Reliability – Resolution + Uncertainty (Murphy, 1973)

Reliability (calibración): gap entre predicho y observado. Menor = mejor.

Resolution (discriminación): variación entre bins. Mayor = mejor.

**Implicación: Brier malo + AUC alto = buen discriminador que necesita calibración.
Z de Spiegelhalter evalúa SOLO calibración → herramienta correcta para decidir.**

La herramienta clave para decidir si calibrar o no. Test de hipótesis para calibración:

$$Z = \frac{\sum_{i=1}^N (\hat{p}_i - y_i)(1 - 2\hat{p}_i)}{\sqrt{\sum_{i=1}^N (1 - 2\hat{p}_i)^2 \hat{p}_i (1 - \hat{p}_i)}}$$

$$|Z| > 1.96 \Rightarrow \text{miscalibrado} \quad (p < 0.05)$$

Interpretación:

- $|Z| < 1.96$: modelo NO significativamente miscalibrado. No tocar.
- $|Z|$ entre 1.96 y 3.0: zona gris. Probar calibración en held-out.
- $|Z| > 3.0$: miscalibración real. Calibrar con Venn-Abers.

Ejemplos del benchmark (21 clasificadores, 30 datasets):

- CatBoost: $|Z| = 1.88 \rightarrow$ ya calibrado. NO calibrar.
- XGBoost: $|Z| = 8.70 \rightarrow$ severamente miscalibrado. Calibrar.
- LightGBM: $|Z| = 5.61 \rightarrow$ miscalibrado. Calibrar.

Regla: medir primero, calibrar solo cuando los datos lo indiquen.

**Los modelos que más querías calibrar son los que menos lo necesitan.
Los modelos que más lo necesitan son los que menos se te ocurre calibrar.**

Evidencia (8 clasificadores, 15,000 muestras, Manokhin 2026):

Modelos que empeoran con calibración post-hoc:

- LightGBM + Isotonic: log-loss +2.2%
- Gradient Boosting + Isotonic: log-loss +8.9% (¡dramático!)
- LR, LDA: también degradados.

Modelos que mejoran con calibración:

- Random Forest + Isotonic: log-loss -14.9%
- Naive Bayes + Isotonic: log-loss -6.2%

¿Por qué?

1. El set de calibración es finito. Para modelos ya calibrados, la "corrección" es ruido → overfitting.
2. Modelos fuertes (CatBoost, LGBM) ya optimizan log-loss internamente → ya están calibrados.
3. Isotonic Regression es codiciosa: con 3000 puntos puede crear cientos de steps → sobreajuste.

Conclusión: "siempre calibrar" es un consejo PELIGROSO.

Ajusta una sigmoide con 2 parámetros (A, B) sobre los outputs del modelo:

$$P(y = 1 \mid f) = \frac{1}{1 + \exp(Af + B)}$$

donde f es el output raw del modelo. A y B se optimizan por MLE en el set de calibración.

Ventajas:

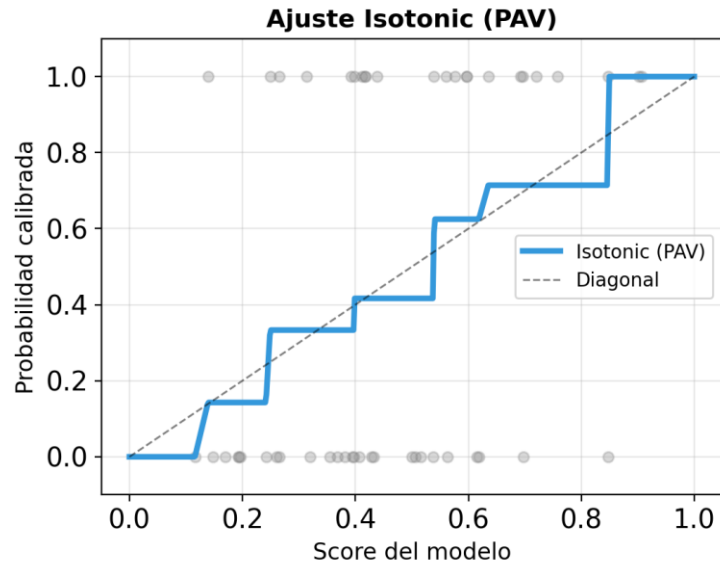
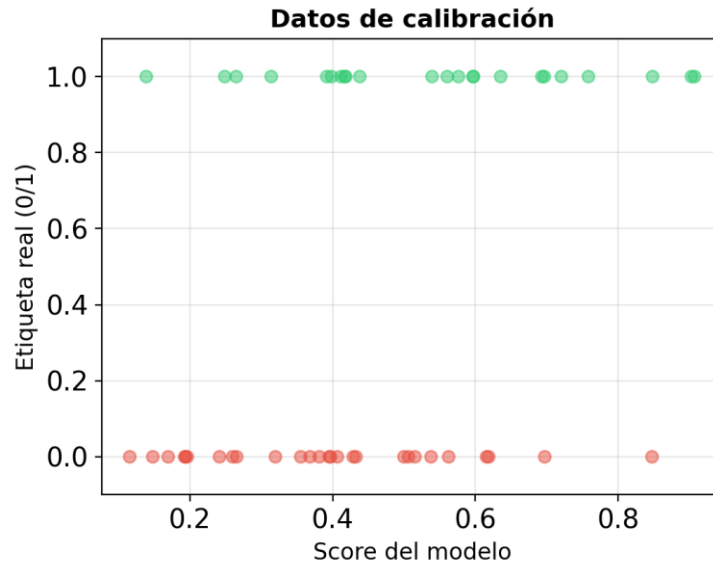
- Simple: solo 2 parámetros → bajo riesgo de overfitting.
- Efectivo para modelos severamente miscalibrados (SVMs, XGBoost, MLP).
- XGBoost: log-loss -8.8%. Random Forest: -14.8%. MLP: -24.3%.

Problemas:

- Asume que log-odds son lineales (relación sigmoide). Si no → falla.
- En el benchmark de 21 clasificadores: mejoró log-loss en solo el 49.8% de los casos. Es un coin flip.
- CatBoost: log-loss +5.3%. Empeoró en el 93% de los folds.
- TabICL: log-loss +6.0%. TabPFN: +5.0%. EBM: +4.4%.
- Comprime las colas: predicciones 0.05 vs 0.85 se acercan → destruye sharpness.

Veredicto: herramienta de rescate para modelos muy mal calibrados. NUNCA usar a ciegas.

Isotonic Regression (PAV)



Algoritmo Pool-Adjacent-Violators (PAV):

1. Ordena scores y asigna etiqueta real (0/1) a cada uno.
2. Donde la monotonía se viola (un 0 después de un 1), fusiona bins y promedia.
3. Resultado: función escalonada monótonamente creciente.

Ventaja: no-paramétrica, captura miscalibraciones complejas.

Problemas :

- Con 3000 puntos: cientos de steps → overfitting si el modelo ya está calibrado.
- $E[\Delta \log\text{-loss}] = +0.21\%$. AUMENTA log-loss en promedio.
- GB + Isotonic: +8.9%. CatBoost degradado sistemáticamente.

Base de Venn-Abers: VA usa isotonic internamente, pero de forma que garantiza calibración.

Método paramétrico que generaliza Platt Scaling con distribución Beta (Kull et al., 2017).

3 parámetros (a , b , m). Incluye la sigmoide como caso particular.

Ventajas:

- Más flexible que Platt: captura formas asimétricas de miscalibración.
- El más consistente: mejoró log-loss en 67.1% de los casos — tasa más alta de los 5 calibradores.
- Coste $\sim 0\%$: $E[\Delta \text{ tiempo inferencia}] = -0.42\%$. Esencialmente gratis.
- $E[\Delta \text{ log-loss}] = -13.70\%$. Segundo mejor tras Venn-Abers.
- $E[\Delta \text{ Brier}] = -3.96\%$. Segundo mejor tras Venn-Abers.

Limitaciones:

- Paramétrica: asume miscalibración en familia Beta. Sin garantía teórica.
- Con modelos ya calibrados (CatBoost, TabICL), puede degradar como Platt.

Cuándo usar:

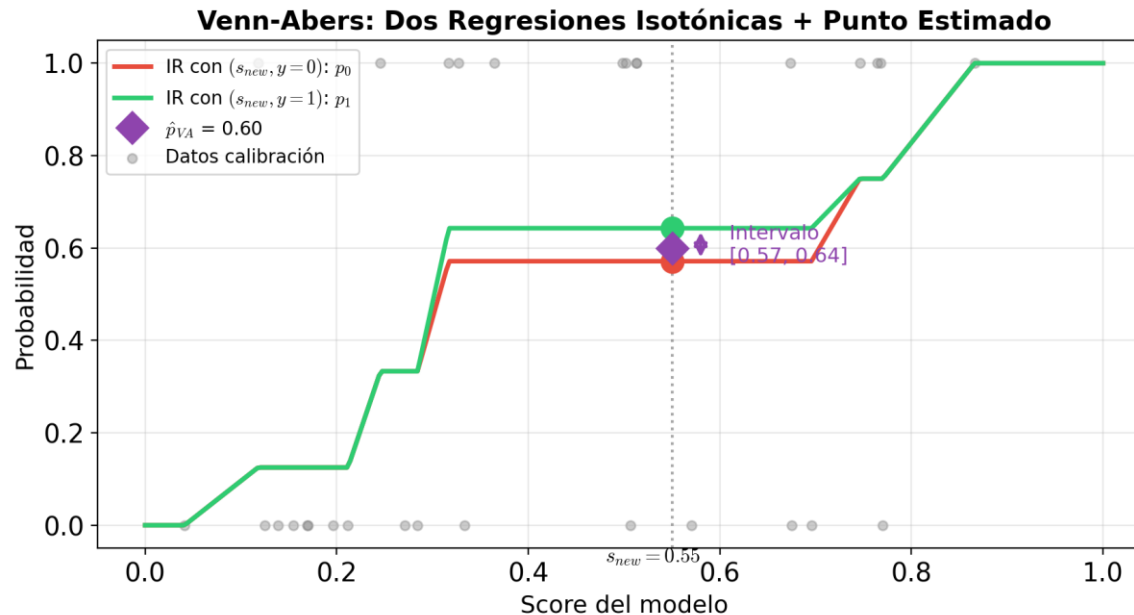
- Cuando Venn-Abers es demasiado lento para producción real-time.
- Como opción pragmática por defecto: mejor ratio beneficio/coste de todos.
- Siempre verificar con $|Z|$ de Spiegelhalter si el modelo necesita calibración.

Librería: `pip install betacal`

Venn-Abers: Cómo Funciona (Detalle)

Para cada punto test con score s_{new} , ejecuta DOS regresiones isotónicas:

1. Añade $(s_{\text{new}}, y=0)$ al set de calibración → ajusta IR → obtiene p_0
2. Añade $(s_{\text{new}}, y=1)$ al set de calibración → ajusta IR → obtiene p_1



La curva roja (hipótesis $y=0$) y verde (hipótesis $y=1$) difieren porque cada una incluye un dato extra distinto.

El intervalo $[p_0, p_1]$ refleja la incertidumbre del modelo. El punto estimado:

$\hat{p}_{VA} = p_1 / (1 - p_0 + p_1) \rightarrow$ balancea ambos extremos.

¿Por qué funciona? Al probar las DOS etiquetas posibles, VA no necesita asumir nada sobre la distribución. La calibración se cumple por construcción bajo intercambiabilidad.

Coste: +139.5% inferencia (2 IR por punto). Set calibración: ≥ 500 muestras.

Procedimiento completo:

1. Dividir datos en Train, Calibración y Test.

2. Entrenar modelo base en Train (cualquier clasificador: XGBoost, RF, NN...).

El modelo produce scores $s(x)$ para cada ejemplo.

3. Calcular scores del set de Calibración: $\{(s_1, y_1), (s_2, y_2), \dots, (s_n, y_n)\}$.

4. Para cada nuevo punto x_{test} :

a) Calcular s_{new} = score del modelo para x_{test} .

b) Crear $\text{set}_0 = \text{calibración} \cup \{(s_{\text{new}}, 0)\}$. Ajustar IR_0 . Evaluar $p_0 = IR_0(s_{\text{new}})$.

c) Crear $\text{set}_1 = \text{calibración} \cup \{(s_{\text{new}}, 1)\}$. Ajustar IR_1 . Evaluar $p_1 = IR_1(s_{\text{new}})$.

d) Calcular: $\hat{p}_{\text{VA}} = p_1 / (1 - p_0 + p_1)$

5. El intervalo $[p_0, p_1]$ mide la incertidumbre epistémica de la calibración.

Intervalo estrecho = alta confianza. Intervalo ancho = más datos de calibración necesarios.

Nota: parece costoso ($n+1$ IR por punto), pero en la práctica el PAV es $O(n)$ y solo se modifica un punto, así que existen implementaciones eficientes.

Garantía: bajo intercambiabilidad, las probabilidades están automáticamente calibradas. No aproximadamente — exactamente. Demostrado por Vovk (2004).

Bajo intercambiabilidad (la misma asunción que subyace a cross-validation), las multi-probabilidades Venn-Abers están automáticamente bien calibradas.

No aproximadamente. No asintóticamente. Automáticamente.

Lo que esto significa:

- Resultado distribution-free: sin asunciones sobre la forma de los datos.
- Sin asunciones sobre el modelo que generó los scores.
- Sin familia paramétrica. Sin tuning.
- La calibración se cumple por construcción.

Ningún otro calibrador post-hoc ofrece esto:

- Platt Scaling: asume log-odds lineales. PUEDE fallar.
- Isotonic Regression: asume monotonía + necesita datos suficientes por bin. PUEDE fallar.
- Temperature Scaling: asume que los logits solo necesitan reescalado. PUEDE fallar.
- Beta Calibration: asume familia beta. PUEDE fallar.
- **Venn-Abers: NO PUEDE fallar bajo intercambiabilidad.**

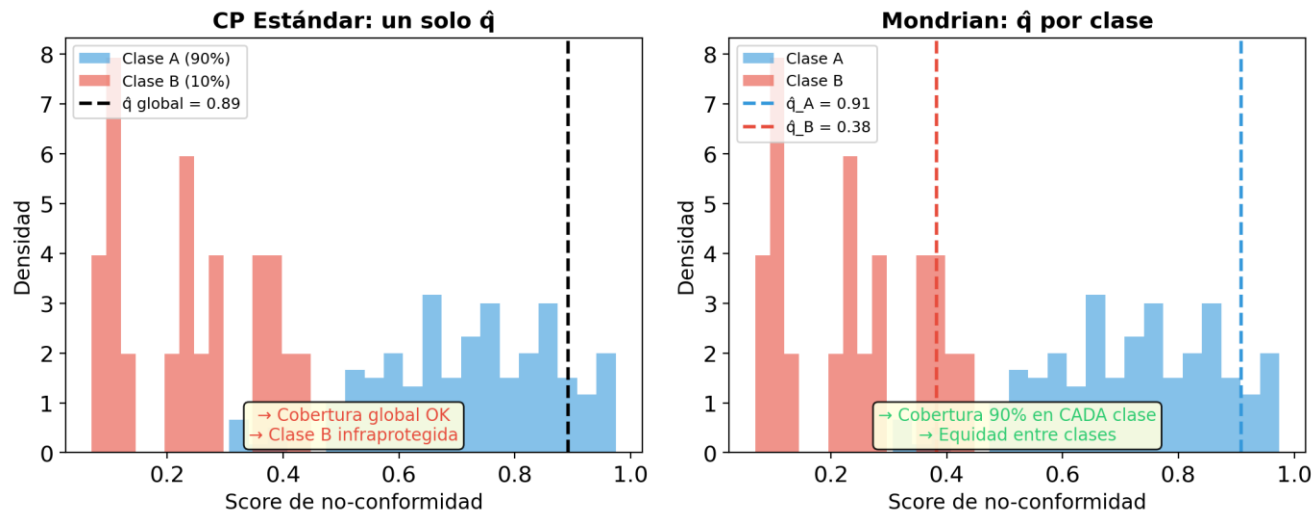
Parte del framework de Conformal Prediction. Misma tradición matemática que los conjuntos de predicción conformales del capítulo anterior.

Mondrian Venn-Abers (Clases Desbalanceadas)

Problema: en CP estándar, un solo $\hat{q}$ global puede dar cobertura 90% global pero 0% en la clase minoritaria.

Mondrian calcula un cuantil de corte diferente por clase:

$$\hat{q}_c = \text{Quantile}\left(\frac{\lceil (n_c + 1)(1 - \alpha) \rceil}{n_c}, \{s_i : y_i = c\}\right) \quad \forall c \in \mathcal{Y}$$



Ventajas:

- Cobertura $\geq 1 - \alpha$ para CADA clase individualmente, no solo en promedio.
- Crucial en datos desbalanceados: medicina (enfermedad rara), fraude ($< 1\%$), defectos industriales.

Es ortogonal al score: se puede combinar con cualquier calibrador.

Mondrian + Venn-Abers = calibración con garantía + equidad entre clases.

Manokhin & Grønhaug (arXiv:2601.19944)

21 clasificadores × 5 calibradores × 38 datasets TabArena-v0.1 × 150 CV folds

Cambio esperado en log-loss (todos los modelos):

- Venn-Abers: -14.17% mejoró 63.2% menos degradación extrema
- Beta calib.: -13.70% mejoró 67.1% el más consistente
- Platt scaling: -9.75% mejoró 49.8% coin flip
- Isotonic reg.: +0.21% aumentó log-loss en promedio
- Pearsonify: +22.89% degradación masiva — no usar

Cambio esperado en Brier score:

- VA: -4.14% | Beta: -3.96% | Isotonic: -3.74% | Platt: -1.96%

AUC-ROC: cambio despreciable (<0.5%). Calibración no mejora discriminación.

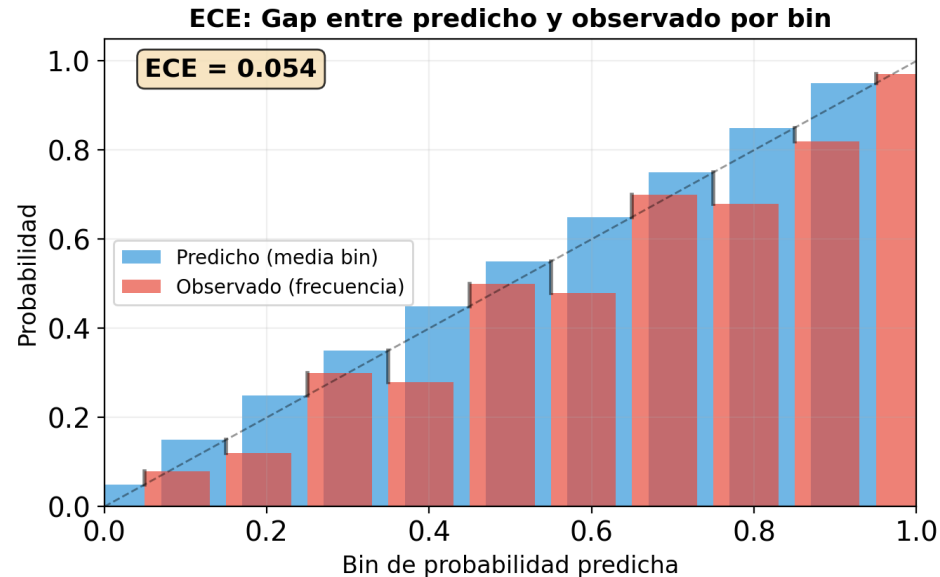
Accuracy: marginal (máx +0.008%). Cambia probabilidades, no predicciones.

Coste inferencia: VA +139.5% | Platt +47.1% | Beta -0.42% | Isotonic -0.93%

Top modelos sin calibrar: TabICL (rank 2.52), CatBoost (3.79), TabPFN (4.24), EBM (4.57).

Estos modelos NO se benefician de calibración post-hoc.

Métricas de Evaluación de Calibración



ECE (Expected Calibration Error):

Media ponderada del $|\text{gap}|$ entre predicho y observado por bin. Fácil de interpretar.
 Problemas: sensible al número de bins. Un modelo que predice siempre la base rate tiene $\text{ECE}=0$ pero es inútil.

Brier Score:

MSE de probabilidades. Proper scoring rule pero menos sensible a errores extremos.
 Descomposición Murphy: Reliability – Resolution + Uncertainty (separa calibración de discriminación).

Spiegelhalter Z: test estadístico puro de calibración. $|Z| > 1.96$ = miscalibrado.

No depende de bins (ventaja sobre ECE). Evalúa SOLO calibración (ventaja sobre Brier/Log-Loss).

Paso 1: Entrenar el modelo.

Paso 2: Calcular $|Z|$ de Spiegelhalter en held-out set.

Si $|Z| < 1.96 \rightarrow$ modelo calibrado. Enviar a producción. NO calibrar.

Modelos típicos: CatBoost, EBM, TablCL, TabPFN.

Si $|Z| > 1.96 \rightarrow$ modelo miscalibrado. Aplicar Venn-Abers.

Modelos típicos: XGBoost, LightGBM, Random Forest, MLP, Naive Bayes.

Si latencia es crítica $\rightarrow$ Beta Calibration como alternativa.

67.1% de mejoras, coste computacional $\sim 0\%$.

Playbook de diagnóstico con Brier Decomposition:

Bajo $|Z|$ + Alto AUC: calibrado y discrimina. Enviar.

Alto $|Z|$ + Alto AUC: mala calibración, buena discriminación. Calibrar con Venn-Abers.

Mejor posición después de "enviar".

Alto $|Z|$ + Bajo AUC: malo en todo. Reentrenar con otro modelo.

Bajo $|Z|$ + Bajo AUC: calibrado pero inútil. Predice la base rate. Cambiar de modelo.

Regla de oro: Descomponer primero. Calibrar selectivamente. Nunca calibrar a ciegas.

venn-abers (paquete Python):

- Implementación oficial de Venn-Abers Predictors.
- 3 líneas de código tras entrenar: split → train → VennAbersCalibrator.
- Set de calibración recomendado: ≥ 500 muestras (menor → intervalos (p_0, p_1) anchos).
- pip install venn-abers

scikit-learn (CalibratedClassifierCV):

- method='sigmoid' → Platt Scaling.
- method='isotonic' → Isotonic Regression.
- Cuidado: el default puede empeorar modelos ya calibrados.
- Útil para: sklearn.calibration.calibration_curve (reliability diagram).

betacal:

- Implementación de Beta Calibration.
- Segunda mejor opción cuando latencia importa (coste $\sim 0\%$).

Diagnóstico (implementación propia con numpy):

- Estadístico Z de Spiegelhalter: fácil de calcular.
- Descomposición de Brier: bins + reliability + resolution.

Referencia: arXiv:2601.19944 — Classifier Calibration at Scale

Libro: Applied Conformal Prediction (V. Manokhin)